

VestiCode 架构设计文档

基于 .NET 10 的自主编码 AI Agent。核心是「思考(Thought) → 行动(Action) → 观察(Observation)」的 ReAct 推理循环，运行于终端。

本文档包含 **Agent 架构图**、**工具设计**、**推理流程图** 三大部分，并展开记忆、安全、Provider、并发模型、多 Agent 等子系统设计。所有结构图均以 HTML 绘制。

1. 项目概述

VestiCode 是一个运行在终端里的通用编码 Agent。用户用自然语言下达编程任务，Agent 通过反复「调用工具 → 观察结果 → 继续推理」自主完成任务：读写文件、执行命令、搜索代码、联网抓取等。

维度	实现
语言 / 运行时	C# / .NET 10 (net10.0)， Nullable 与 TreatWarningsAsErrors 全开
LLM 接入	HTTP 直连 + SSE 流式解析，适配 OpenAI / Anthropic 两套线缆协议（DeepSeek 走 OpenAI 兼容协议）
交互界面	控制台 TUI：行内流式渲染、Markdown、状态行、人在回路(HITL)弹窗
架构风格	Clean Architecture：领域核心 (Core) 与界面 (Cli) 彻底解耦，经由事件流通信
核心能力	ReAct 循环、Function Calling、流式输出、记忆与压缩、安全门控、MCP 客户端、Skill、Hook、子 Agent、多 Agent 团队、git worktree 隔离

2. 解决方案分层 (Clean Architecture)

领域核心不依赖任何 UI；界面层只负责把核心产出的「事件流」渲染成终端画面。这使同一套领域逻辑既能被任意前端复用，也能被单元测试独立验证。

项目	职责	依赖方向
VestiCode.Core	领域核心：Agent 循环、Provider 抽象、工具体系、记忆、安全、MCP、Skill、Hook、子 Agent、团队、worktree。 不含任何 UI 代码。	仅 .NET BCL + 少量库（YamlDotNet 等）
VestiCode.Cli	宿主与界面：Generic Host + 依赖注入 + 分层配置 + Serilog 日志 + 控制台 TUI。消费 Core 的事件流并渲染。	→ VestiCode.Core
VestiCode.UnitTests	xUnit 单元测试（58 个），覆盖 Agent 循环、安全、MCP、Skill、Hook、子 Agent、团队、会话、压缩等核心组件。	→ VestiCode.Core

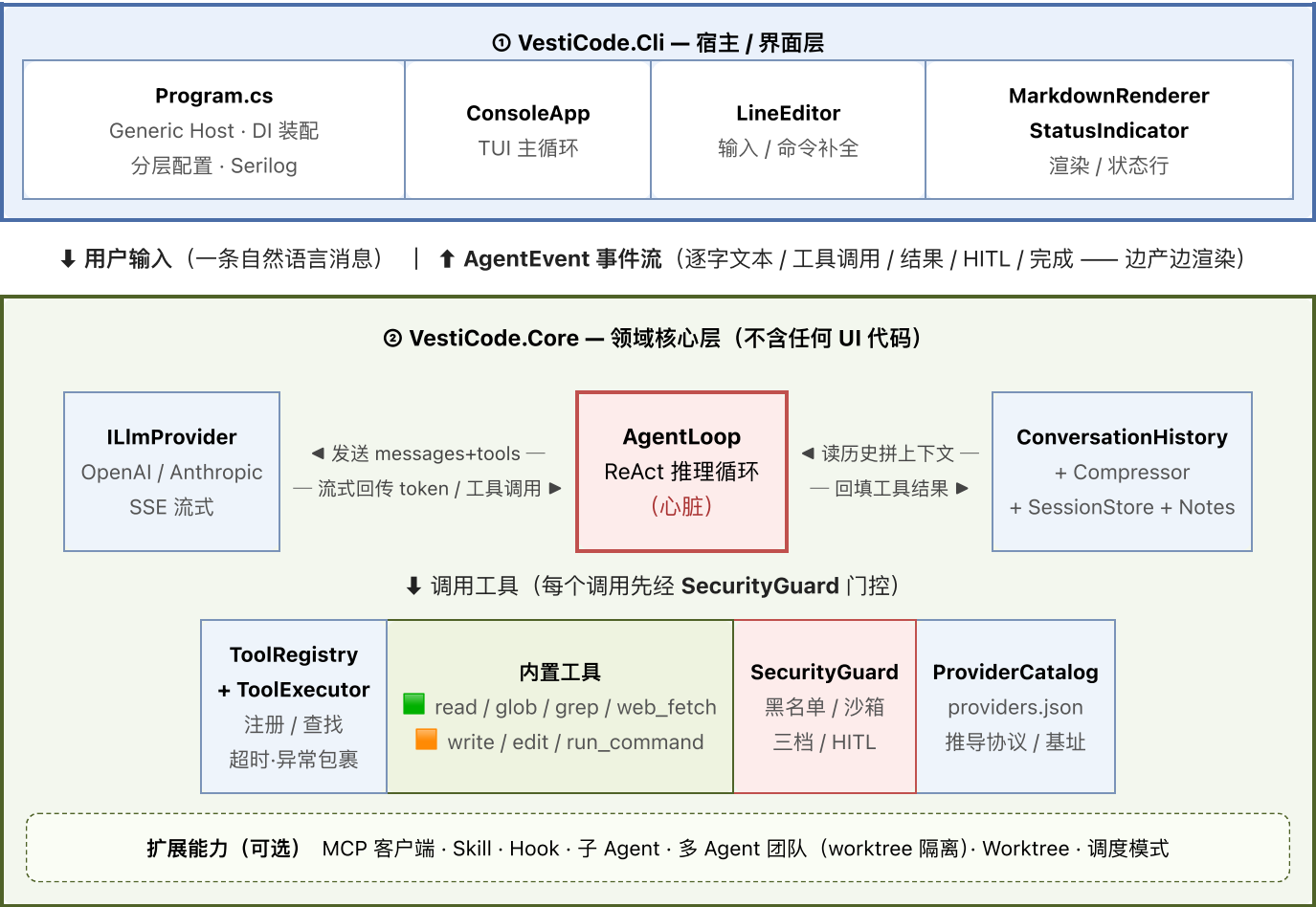
3. Agent 核心架构 (四大组件)

一个 AI Agent 由四个核心组件构成，本项目的落点如下。其中 **Agent Loop** 是 **Agent** 区别于普通 LLM 对话的关键——它让模型能反思考、行动、观察，自我驱动直至任务完成。

组件	角色	本项目实现
LLM (大脑)	推理与决策	ILlmProvider + OpenAIProvider / AnthropicProvider (SSE 流式)
Agent Loop (控制循环)	驱动 ReAct 迭代	AgentLoop.RunAsync (async IAsyncEnumerable<AgentEvent>)
Memory (记忆)	维持上下文	ConversationHistory + JsonLSessionStore + ContextCompressor + AutoNoteManager
Tools (工具)	与外界交互	ITool + ToolRegistry + ToolExecutor (7+ 内置 + 动态 MCP)

3.1 总体架构图

图例：■ 核心循环 · ■ 宿主 / 普通组件 · ■ 只读 · ■ 写入 · 虚线框 = 可选扩展





流程层面的关键设计：

设计	说明
流式即可观测	每个 token / 思考 / 工具调用 / 工具结果都实时以事件产出，推理过程对用户完全透明
读并发 / 写串行	只读工具无副作用 -> Task.WhenAll 并发加速；写类有副作用 -> 串行，避免文件竞态
Observation 必回填	每个工具结果作为 tool 消息写回历史，模型下一轮才能看到"结果继续推理 (ReAct 闭环)
双重终止条件	模型不再请求工具 (NoToolCall) 或达到 MaxRounds
消息配对约束	必须先写 assistant 的 tool_use，再写对应 tool 结果，否则协议不配对会被 API 拒绝

5. 工具设计（Tool Calling）

5.1 工具抽象与契约

所有工具实现统一接口 `ITool`；`ToolRegistry` 负责注册与查找，并把工具元数据导出为 LLM 可理解的 Function Calling 定义；`ToolExecutor` 统一包裹执行（超时、异常转 `ToolResult.Fail`、日志）。



执行时：`ToolExecutor.ExecuteAsync` 统一包裹 超时 / 异常 / 日志 -> 返回 `ToolResult` (Success + Output / Error)

`ITool.Category` 默认值为 **Write**（保守默认），只读工具显式覆盖为 **Read**。这一默认值保证"未声明类别的工具一律按写入处理"，安全优先。

5.2 内置工具清单

工具	类别	作用	并发性
<code>read_file</code>	Read	读取文件内容（去除 UTF-8 BOM）	可并发
<code>glob</code>	Read	通配符匹配文件路径	可并发

grep	Read	正则搜索文件内容，返回 文件:行号:内容	可并发
web_fetch	Read	抓取 URL 内容	可并发
write_file	Write	新建文件（已存在则拒绝，提示改用 edit）	串行
edit_file	Write	精确字符串替换（old_string 须在文件中唯一）	串行
run_command	Write	持久 shell 执行命令（cd / 环境变量跨调用保持）	串行
sub_agent	Write	派生子 Agent 处理子任务（run-to-end）	串行
{server}__{tool}	Read/Write	MCP 动态工具（运行期从外部 server 发现注册）	按类别

Category 的双重作用：① 决定主循环"读并发 / 写串行"的分批；② 决定安全门控的默认放行策略（读放行、写询问）。

5.3 工具调用全链路

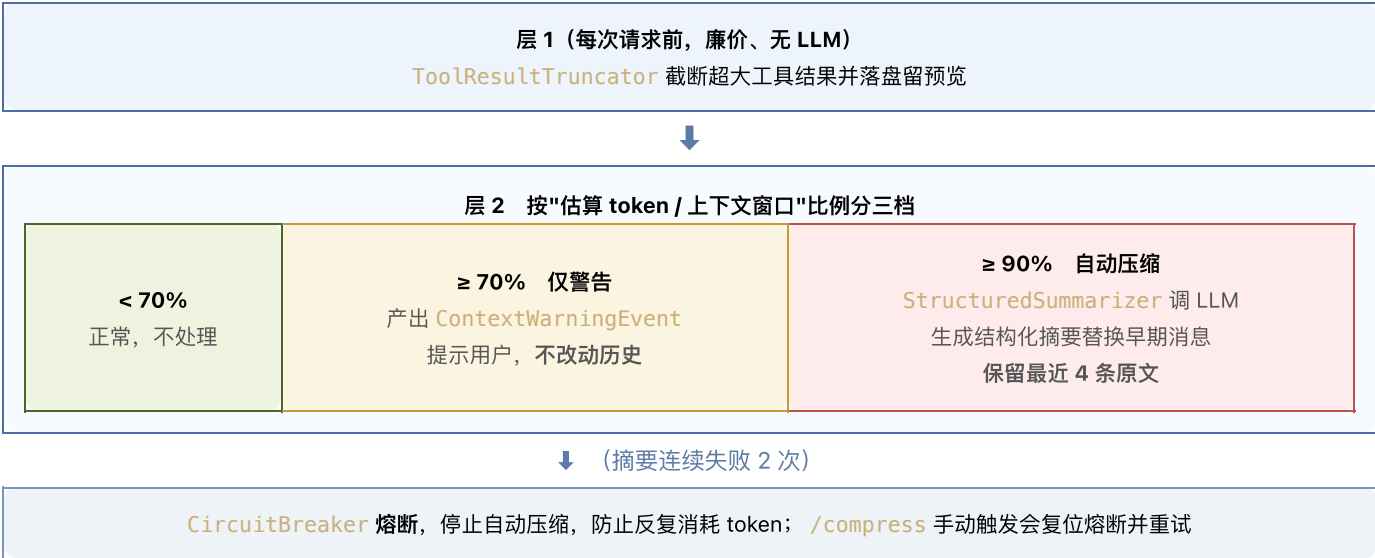


6. 记忆架构（Memory）

VestiCode 实现了三层记忆 + 两段式压缩 + 四类长期笔记。

层级	实现	说明
短期记忆	ConversationHistory	当前会话完整消息列表（System / User / Assistant / Tool 四种角色）
工作记忆	工具结果回填 + 每轮注入	中间状态（工具 Observation、plan-only 提醒等）随历史滚动
持久记忆	JsonlSessionStore	会话落盘 {id}.jsonl + {id}.meta.json；支持 --resume 恢复、跳过损坏行、未配对 tool_use 处截断、>30 分钟注入时间提醒
长期偏好	AutoNoteManager	每 5 轮按四类（用户偏好 / 纠正反馈 / 项目知识 / 参考资料）增量提炼笔记，跨会话可用

6.1 两段式上下文压缩



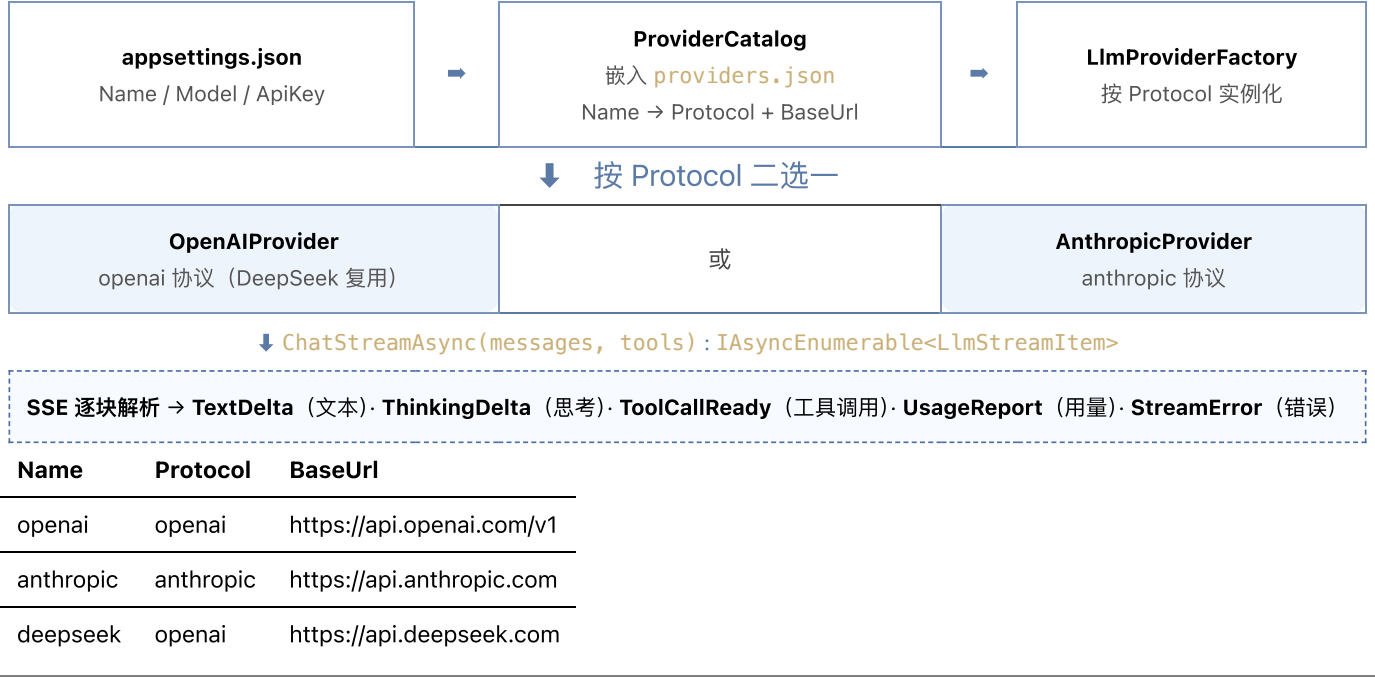
7. 安全架构（边界控制）

SecurityGuard 在每个工具执行前进行多层纵深防御。



8. Provider 抽象与流式

用户只需配置 Name / Model / ApiKey；协议与基址由内置的 providers.json 推导，新增后端只需追加一行配置、无需改代码。



9. 并发模型

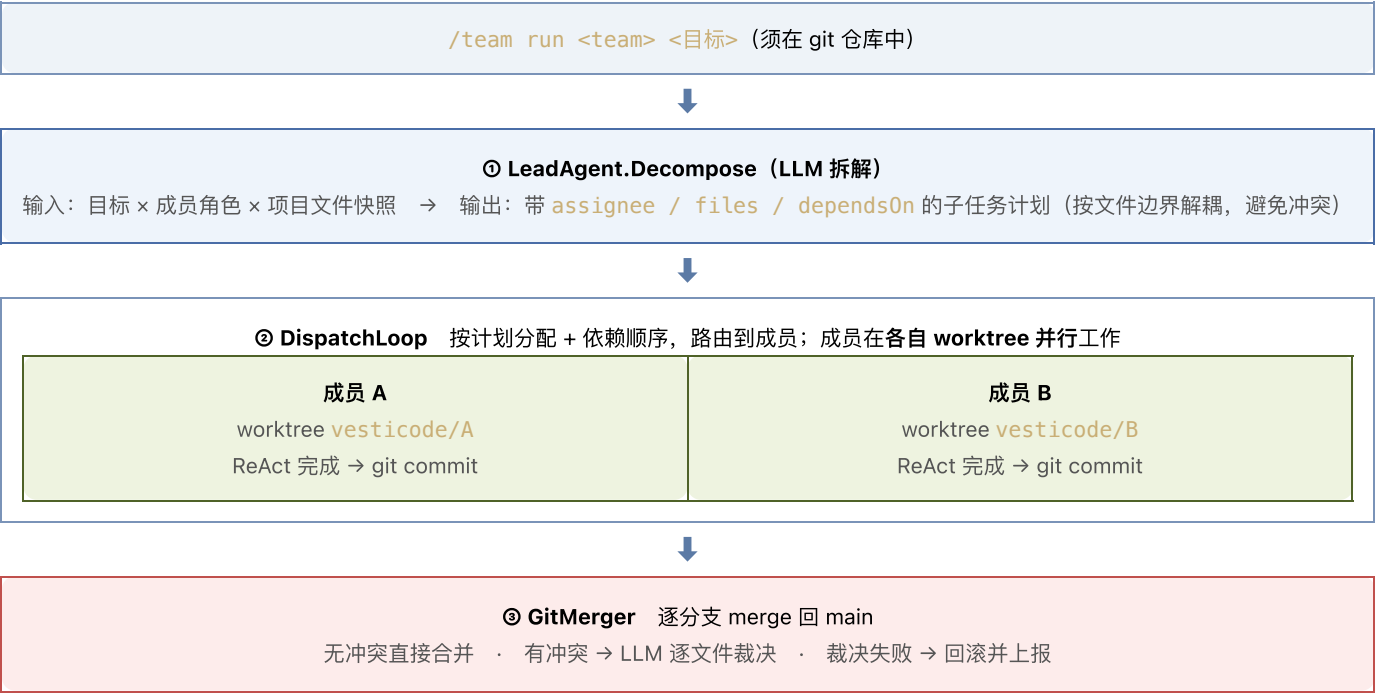
VestiCode 的并发集中在 Agent 循环与工具执行，采用「读并发 / 写串行」并辅以多种异步原语。

机制	用途	原语
异步流	LLM SSE 与 Agent 事件边产边消费	<code>IAsyncEnumerable<T> + await foreach</code>
读类并发	同一轮多个只读工具并行	<code>Task.WhenAll</code>
写类串行	避免写工具相互竞态	顺序 <code>foreach + await</code>
Shell 串行	持久 shell 命令排队	<code>SemaphoreSlim</code>
人在回路	挂起等待用户授权而不阻塞线程	<code>TaskCompletionSource</code> (<code>RunContinuationsAsynchronously</code>)
取消	轮次边界优雅停止 / 流中途中断	<code>CancellationToken + [EnumeratorCancellation]</code>
团队并行	多成员各自 worktree 物理隔离并行	git worktree + 进程级文件隔离

10. 高级模块

模块	能力	关键类型
MCP (客户端)	stdio / HTTP 连接外部 MCP server，动态注册工具/资源/提示词（首次调用惰性发现并缓存）；工具命名 <code>{server}__{tool}</code>	<code>McpManager</code> / <code>McpClient</code> / <code>McpAdapters</code>
Skill	两阶段技能：阶段 1 描述注入 → 阶段 2 SOP 钉入；可声明工具白名单约束 Agent 可见工具	<code>SkillRegistry</code> / <code>SkillTool</code>
Hook	生命周期事件钩子（轮次、工具 pre/post），可拦截或记录；YAML 配置，加载期校验	<code>HookEngine</code> / <code>HookLoader</code>
子 Agent	run-to-end 子任务执行；角色化工具过滤（如 explorer 仅 read/glob/grep）；全局禁止递归创建	<code>SubAgentRunner</code> / <code>RoleLoader</code> / <code>ToolFilter</code>
多 Agent 团队	Lead 用 LLM 拆解目标 → 成员各自 git worktree 隔离工作 → 合并回 main（冲突 LLM 逐文件裁决，失败回滚）	<code>LeadAgent</code> / <code>TeamMember</code> / <code>GitMerger</code>
Worktree	git worktree 隔离工作区：create / enter / merge / exit(非破坏性离开) / remove	<code>GitWorktreeManager</code>
调度模式	双锁激活后剥夺 Lead 的读写/执行工具，注入 10 阶段指挥官 workflow，迫使其只委派	<code>DispatchScheduler</code>

10.1 多 Agent 团队流程





© LeadAgent.Synthesize (LLM 综合) 汇总各成员产出，给出最终报告

11. .NET 技术深度

.NET 特性	用法
依赖注入 (DI)	Microsoft.Extensions.DependencyInjection: 所有组件构造注入；可选依赖用可空构造参数（如 AgentLoop 的 securityGuard / compressor / promptBuilder ...）
Generic Host	Host.CreateApplicationBuilder 统一服务装配与生命周期
配置系统	分层叠加：全局 / 项目 JSON → VESTICODE_CONFIG 指定文件 → 环境变量 → UserSecrets（密钥不入库）
日志	Serilog 滚动文件日志（~/vesticode/logs/）+ 构建期自举 logger
异步流	async IEnumerable<T> + await foreach 贯穿 LLM 流式与 Agent 事件流
并发原语	Task.WhenAll / SemaphoreSlim / TaskCompletionSource / CancellationToken
记录类型	record 建模不可变事件/消息（AgentEvent / ChatMessage / ToolResult）
可空引用类型	全程 Nullable 开启，TreatWarningsAsErrors 强约束

12. 事件驱动设计（UI 解耦）

AgentLoop 不直接操作终端，而是产出 AgentEvent，由 ConsoleApp 翻译为画面。同一套领域逻辑因此可换任意 UI。

事件	含义	TUI 呈现
RoundStartEvent	新一轮开始	内部计数
TextDeltaEvent	回复文本增量	逐字流式打印 + Markdown 渲染
ThinkingEvent	推理/思考增量	灰色 Thinking / Reasoning
ToolCallEvent	请求调用工具	● tool(args)
ToolResultEvent	工具返回（携带本次 Arguments）	多行结果预览
ToolBlockedEvent	被安全/Hook/plan-only 拦截	拦截原因提示
HitlRequestEvent	需用户授权	A/S/P/D 弹窗（TaskCompletionSource 回传决定）
UsageEvent	token 用量上报	状态行计数
ContextWarningEvent / CompactionEvent	上下文 70% 警告 / 90% 压缩	灰色提示行
AgentDoneEvent	循环结束（NoToolCall / MaxRounds / Cancelled）	收尾
ErrorEvent	LLM 流不可恢复错误	红色错误，不崩溃

13. 目录结构

路径	职责
src/VestiCode.Core/Agents/	AgentLoop（推理循环·心脏）、AgentEvent

src/VestiCode.Core/Llm/	ILlmProvider、 OpenAI/Anthropic Provider、 ProviderCatalog、 ChatMessage
src/VestiCode.Core/Tools/	ITool、ToolRegistry、 ToolExecutor、Builtin/* (内置工具)
src/VestiCode.Core/Conversation/	ConversationHistory、 ContextCompressor、 StructuredSummarizer、 CircuitBreaker、 ToolResultTruncator
src/VestiCode.Core/Memory/	JsonlSessionStore (会 话持久化)
src/VestiCode.Core/Security/	SecurityGuard、 CommandBlacklist、 PathSandbox、 SecurityPolicy
src/VestiCode.Core/{Mcp,Skills,Hooks,SubAgents,Teams,Worktree,Notes,Prompts}/	各高级模块
src/VestiCode.Cli/Program.cs	宿主 + DI + 分层配置 + 日志装配
src/VestiCode.Cli/Tui/	ConsoleApp、 LineEditor、 MarkdownRenderer、 StatusIndicator
tests/VestiCode.UnitTests/	58 个 xUnit 测试